PROJE 08 / 20 · ÇALIŞAN TEMEL SÜRÜM

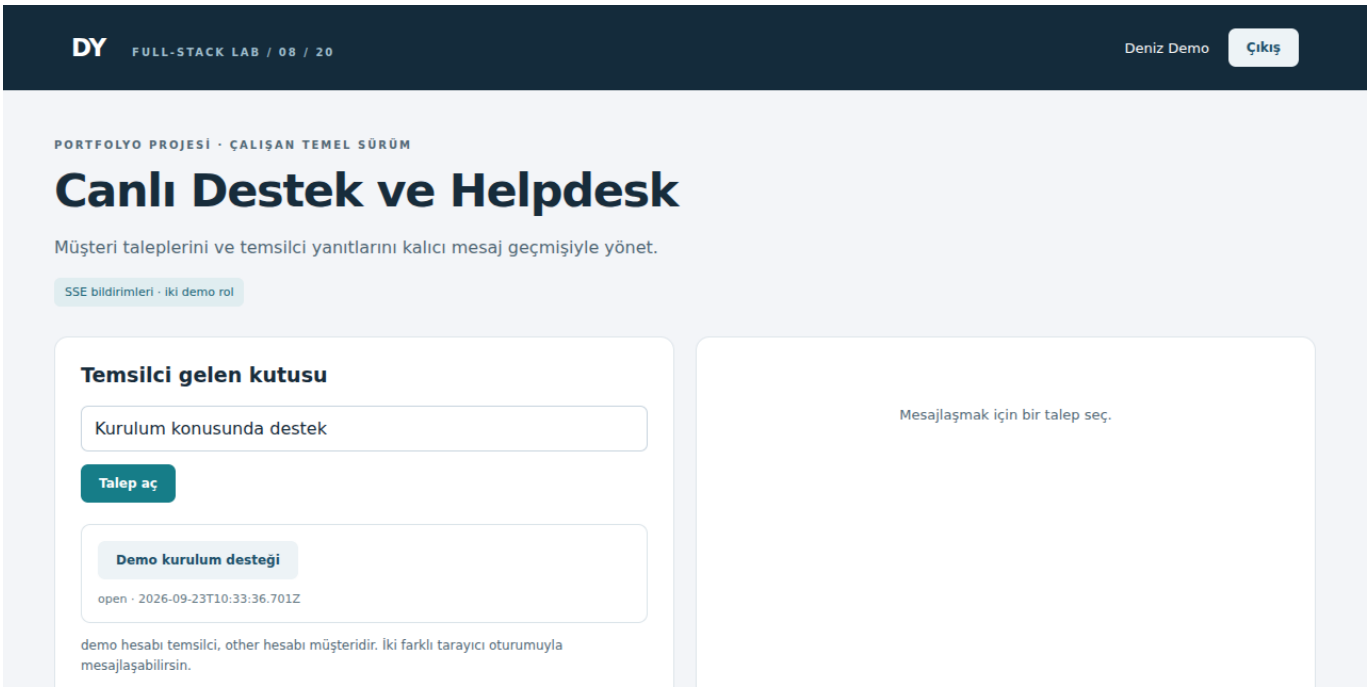
Canlı Destek ve Helpdesk

Müşteri taleplerini ve temsilci yanıtlarını kalıcı mesaj geçmişiyle yönet.

SSE bildirimleri · iki demo rol

Bu proje nedir?

Frontend kullanıcının gördüğü ekranları ve etkileşimleri, backend iş kurallarını ve veri erişimini yürütür. Bu uygulama ikisini aynı origin üzerinde birleştiren bağımsız bir full-stack örnektir. Üretim ürünü değil, üzerinde geliştirme yapabileceğin çalışan başlangıçtır.



Gerçek uygulama ekranı · yerel demo verileri

Teslim edilen kapsam

Talep açma · Temsilci gelen kutusu · Kullanıcı mesajları · SSE ile yenileme · Açık/beklemede/kapalı durumları

Bu sürümde olmayanlar

demo hesabı temsilci, other hesabı müşteridir; genel rol yönetim ekranı yok. Chatbot, e-posta ve dosya eki yok. Mesajlar HTTP POST ile kaydedilir, SSE değişikliği duyurur.

02 / Backend ve veri modeli

Talep sahibi veya demo temsilcisi mesaj okuyup yazabilir. Mesajların artan integer kimliği after parametresiyle geçmiş tamamlama sağlar. SSE mesaj içeriğini değil değişen ticket kimliğini taşır; istemci yetkili API'den geçmişi çeker. Kapalı talebe yeni mesaj 409 döner; yalnız temsilci durumu değiştirir.

Gerçek SQL şeması

```
CREATE TABLE tickets(id TEXT PRIMARY KEY,owner TEXT REFERENCES users(id),subject
TEXT,status TEXT DEFAULT 'open',created_at TEXT);


 CREATE TABLE messages(id INTEGER PRIMARY KEY,ticket TEXT REFERENCES tickets(id),author
TEXT REFERENCES users(id),body TEXT,created_at TEXT);
```

Tam şema: schema.sql. users ve sessions ortak kimlik tablolarıdır. owner/user alanları kullanıcıyı, PRIMARY KEY kaydı, UNIQUE tekilliği ve REFERENCES ilişkiyi belirtir. Para kullanılan projelerde INTEGER kuruş, zaman alanlarında ISO metin veya epoch kullanılır.

Kod okuma sırası

Dosya	Ne arayacaksın?
client/Workspace.vue	Eylemi tetikleyen form ve API isteği
client/api.ts	JSON isteği, hata yakalama ve mesaj
server/project.js	Yetki kontrolü → kural → SQL → yanıt
server/core.js	Parametreli SQL, transaction, oturum, SSE

Neden bu mimari?

Aynı dilde frontend/backend öğrenmeyi kolaylaştırır. Node.js 24 ve Express HTTP katmanıdır; Vue arayüzü yönetir. SQLite ayrıca sunucu kurmadan kalıcılık sağlar. Senkron veritabanı işlemleri küçük yerel demo için seçildi; çok süreçli, yüksek trafikli üretim mimarisi iddiası yok.

03 / API sözleşmesi

Aşağıdaki yollar doğrudan teslim edilen server/project.js dosyasından çıkarılmıştır. JSON alan adları kodla aynıdır. :id ve benzeri path alanları oluşturma yanıtından alınmalıdır. Liste/görünüm uçlarında 200, yeni kayıt uçlarında 201, kuyruk kabulünde 202 kullanılır.

Yöntem / yol	Girdi alanları	Başarı
GET /api/tickets	Gövde yok	200
POST /api/tickets	JSON: subject	201
GET /api/tickets/:id/messages	Query: after	200
POST /api/tickets/:id/messages	JSON: body	201
PATCH /api/tickets/:id	JSON: status	200

Somut istek örneği

```
POST /api/tickets
{"subject":"Kurulum desteği"}
201 {"id":"<ticket-id>"}
```

Oturum ve hata biçimi

POST /api/auth/login: email ve password. Başarılı giriş HttpOnly çerez verir. GET /api/auth/me kullanıcıyı, POST /api/auth/logout çıkışı yönetir. Tarayıcı aynı origin kullanır. Genel hata biçimi: {"error":"açıklama"}. Yetkisiz alan işlemleri sunucuda engellenir; frontend kontrolü tek başına yeterli değildir.

Kod	Anlam
401 / 403	Giriş gerekli / rol veya origin reddedildi
404 / 409	Kayıt görünmüyor / durum veya benzersizlik çakışması
422 / 429	Geçersiz girdi / giriş veya uygulama kotası
500 / 502 / 503	Beklenmeyen hata / dış servis hatası / kapalı veya ayarsız servis

Tüm uçlar bütün kodları üretmez. İstek/yanıt gövdelerini tests/project.test.js ve scripts/seed.js içindeki çalışan örneklerle karşılaştır.

04 / Test ve kabul kriterleri

Bir projenin çalışması yalnız ekrana veri gelmesi değildir. Yetki, tekrar istek, yanlış girdi ve yarış koşulları da beklenen sonucu üretmelidir. Aşağıdaki maddeler teslim edilen temel sürümün kontrol listesidir.

- Müşteri ve temsilci aynı talepte mesajlaşabilmeli.
- Müşteri talep durumunu değiştirememeli.
- Kapalı talebe mesaj 409 dönmeli.
- Mesajlar sayfa yenilendikten sonra da görünmeli.

Otomatik test nasıl çalışır?

npm test, Node test runner ile gerçek HTTP sunucusunu rastgele yerel portta başlatır. İki kullanıcıyla oturum açar, uç noktalara istek gönderir ve durum/yanıtları assert ile denetler. Veritabanı :memory: olduğu için kendi data/app.sqlite dosyan etkilenmez. Projeye özgü testler tests/project.test.js içindedir.

```
npm ci
npm run typecheck
npm test
npm run build
```

Test kodundan örnek

```
const t=await call('/api/tickets','POST',{subject:'Help'},201,other);
await call('/api/tickets/'+t.id+'/messages','POST',{body:'Hello'},201,other);
await post('/tickets/'+t.id+'/messages',{body:'Support reply'},201);
assert.equal((await
call('/api/tickets/'+t.id+'/messages','GET',undefined,200,other)).length,2);
await call('/api/tickets/'+t.id,'PATCH',{status:'closed'},403,other);
```

Tarayıcıda kontrol et

Bir başarılı işlem, bir hatalı girdi ve boş durum göster. 390 px mobil görünümde yatay taşmayı kontrol et. Sayfayı yenilediğinde kalıcı kayıtlar durmalı. Network panelinde isteğin gövdesini ve dönen HTTP kodunu oku; başarısız işlemi başarılıymış gibi göstermemelisin.

Doğrulamanın sınırı

API entegrasyon testleri, üretim derlemesi ve Vue tip/şablon kontrolü uygulandı. Bunlar yük testi, penetrasyon testi veya tüm tarayıcılarda kapsamlı uçtan uca test yerine geçmez.

05 / Çalışma ve GitHub planı

İlk günün somut hedefi

Normal ve gizli pencerede iki hesabı aç. Müşteri talep oluştursun, temsilci yanıtlasın; sayfa yenilemeden mesaj geldiğini göster.

Çalışma bloğu	Teslim edeceğin çıktı
1 · Kurulum / 30-60 dk	Çalışan ekran, örnek veri ve ilk başarılı API isteği
2 · Kod okuma / 45-90 dk	Formdan SQL sorgusuna kadar bir isteğin açıklaması
3 · Geliştirme / 60-120 dk	Bir iş kuralı veya arayüz davranışında kendi değişikliğin
4 · Doğrulama / 30-60 dk	İlgili test ve README değişikliği; yeni ekran görüntüsü

Süreler tahmindir; hata çözme ve yeni kavramlar daha uzun sürebilir. Bu plan hazır temel sürüm üzerinde çalışmak içindir. Projeyi sıfırdan ve üretim kalitesinde bir günde bitirme vaadi değildir.

İleri sürüm: dört gerçek geliştirme

- 1. Rol tablosu ve temsilci atama ekle
- 2. Mesaj sayfalama ve okundu durumu ekle
- 3. Bağlantı yenileme stratejisini genişlet
- 4. Dosya eki ve otomatik yönlendirme ekle

Anlamlı commit sırası

```
git add .
git commit -m "chore: add helpdesk-chat baseline"
# Kendi değişikliğini yaptıktan sonra:
git add client server tests README.md
git commit -m "feat: describe your actual improvement"
```

Yeni boş GitHub reposuna remote ekleyip push et. Burada gerçek hesap/repo adresi bulunmadığı için otomatik yükleme yapılmadı. node_modules, data ve .env dosyalarını hariç tut. Her gün bir gerçek ilerleme paylaş; commit sayısını artırmak için geçmiş üretme.

Mülakatta anlat

Mesajı doğrudan socket üzerinden saklamak yerine HTTP + SSE kullanmanın artıları ve eksileri nelerdir?

Özel kurulum / ilerleme notu

Önce yerel sürümü tamamla. Sonraki aşamada PostgreSQL, migration, pagination ve gerçek kullanıcı yönetimini ihtiyaca göre ekle. Ortak kurulum ayrıntıları README.md ve başlangıç rehberindedir.